



HOA SEN
UNIVERSITY

Lecture 12

Functional Dependencies

Objectives

- Informal Design Guidelines for Relational Databases
 - Semantics of the Relation Attributes
 - Redundant Information in Tuples and Update Anomalies
 - Null Values in Tuples
 - Spurious Tuples
 - Functional Dependencies (FDs)
 - Definition of FD
 - Inference Rules for FDs
 - Equivalence of Sets of FDs
 - Minimal Sets of FDs
 - Key of Relational Schema
-
- Reference: Chapter 14 &15

- Two levels of relation schemas
 - The logical "user view" level – how users interpret the relation schemas and the meaning of their attributes
 - The storage "base relation" level – how tuples in a base relation are stored and updated.

Informal Design Guidelines for Relational Databases (2)

- What is relational database design?
 - The grouping of attributes to form "good" relation schemas
- Design is concerned mainly with base relations
- What are the criteria for "good" base relations?

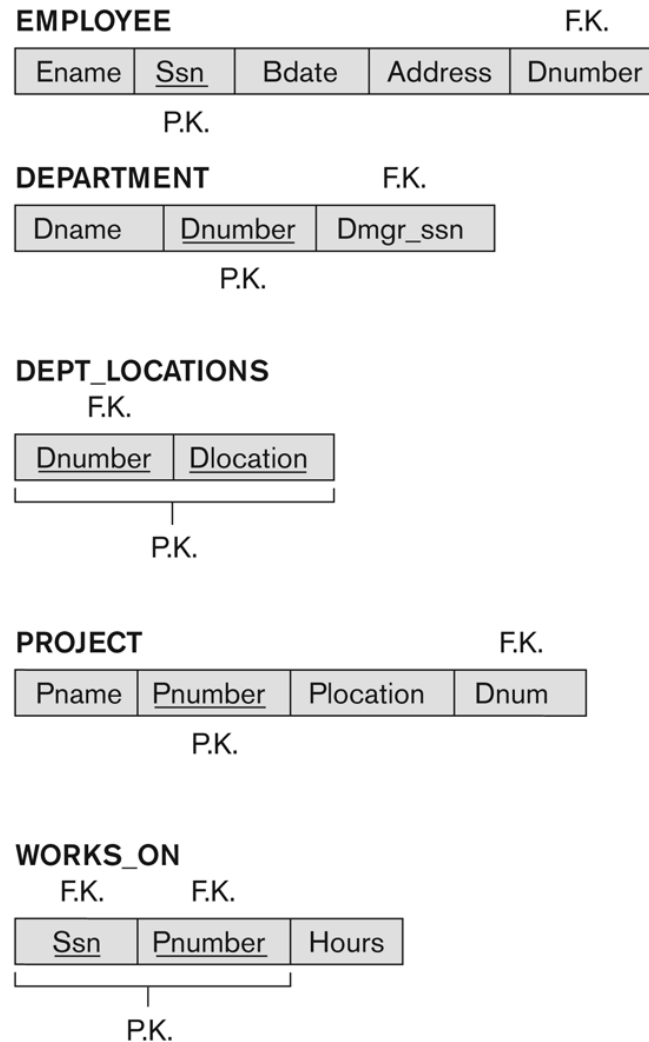
Informal Design Guidelines for Relational Databases (3)

- We first discuss informal guidelines for good relational design
- Then we discuss formal concepts of functional dependencies and normal forms
 - - 1NF (First Normal Form)
 - - 2NF (Second Normal Form)
 - - 3NF (Third Normal Form)
 - - BCNF (Boyce-Codd Normal Form)

Semantics of the Relation Attributes

- **GUIDELINE 1:** Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).
 - Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
 - Only foreign keys should be used to refer to other entities
 - Entity and relationship attributes should be kept apart as much as possible.
- **Bottom Line:** *Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.*

A simplified COMPANY relational database schema



Redundant Information in Tuples and Update Anomalies

- Information is stored redundantly
 - Wastes storage
 - Causes problems with update anomalies
 - Insertion anomalies
 - Deletion anomalies
 - Modification anomalies

Example Of An Insert Anomaly

- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Insert Anomaly:
 - Cannot insert a project unless an employee is assigned to it.
- Conversely
 - Cannot insert an employee unless an he/she is assigned to a project.

Example

- Insert project: **ProductA** → what employee?
- Insert employee: **Jack, Billy** → what project?

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Example Of An Delete Anomaly

- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Delete Anomaly:
 - When a project is deleted, it will result in deleting all the employees who work on that project.
 - Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

Example

- Delete Product Y → Which tuples will be deleted?

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Example Of An Update Anomaly

- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Update Anomaly:
 - Changing the name of project number P1 from “Billing” to “Customer-Accounting” may cause this update to be made for all 100 employees working on project P1.

Example

- Changing name of project “Product Y” → “Jean”

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Guideline to Redundant Information in Tuples and Update Anomalies

- **GUIDELINE 2:**

- Design a schema that does not suffer from the insertion, deletion and update anomalies.
- If there are any anomalies present, then note them clearly and make sure that programs that update the database will operate correctly.
- The 2nd guideline is consistent with and a restatement of the 1st guideline.

Null Values in Tuples

- Reasons for nulls:
 - Attribute not applicable or invalid
 - Attribute value unknown (may exist)
 - Value known to exist, but unavailable
- **GUIDELINE 3:**
 - Relations should be designed such that their tuples will have as few NULL values as possible
 - Attributes that are NULL frequently could be placed in separate relations (with the primary key)

Spurious Tuples

- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The "lossless join" property is used to guarantee meaningful results for join operations
- **GUIDELINE 4:**
 - The relations should be designed to satisfy the lossless join condition.
 - No spurious tuples should be generated by doing a natural-join of any relations.

Spurious Tuples (2)

- There are two important properties of decompositions:
 - a) Non-additive or losslessness of the corresponding join
 - b) Preservation of the functional dependencies.
- Note that:
 - Property (a) is extremely important and *cannot* be sacrificed.
 - Property (b) is less stringent and may be sacrificed.

Functional Dependencies

- **Functional dependencies (FDs)**
 - Are used to specify *formal measures* of the "goodness" of relational designs
 - And keys are used to define **normal forms** for relations
 - Are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes
- A set of attributes X *functionally determines* a set of attributes Y if the value of X determines a unique value for Y

Functional Dependencies (2)

- **Definition:**

$X \rightarrow Y$ holds if whenever two tuples have the same value for X , they *must have* the same value for Y

- For any two tuples t_1 and t_2 in any relation instance $r(R)$:
If $t_1[X]=t_2[X]$, *then* $t_1[Y]=t_2[Y]$

- $X \rightarrow Y$ in R specifies a *constraint* on all relation instances $r(R)$
- FDs are derived from the real-world constraints on the attributes

Examples of FD constraints

- Social security number determines employee name
 - $SSN \rightarrow EName$
- Project number determines project name and location
 - $PNumber \rightarrow \{PName, PLocation\}$
- Employee SSN and project number determines the hours per week that the employee works on the project
 - $\{SSN, PNumber\} \rightarrow Hours$

Examples of FD constraints (2)

- An FD is a property of the attributes in the schema R

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

- The constraint must hold on *every* relation instance $r(R)$
- If K is a key of R , then K functionally determines all attributes in R

Inference Rules for FDs

- Given a set of FDs F , we can **infer** additional FDs that hold whenever the FDs in F hold
- **Armstrong's inference rules:**
 - IR1 – **Reflexive**: If Y *subset-of* X , then $X \rightarrow Y$
 - IR2 – **Augmentation**: If $X \rightarrow Y$, then $XZ \rightarrow YZ$
 - (Notation: XZ stands for $X \cup Z$)
 - IR3 – **Transitive**: If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$
- IR1, IR2, IR3 form a **sound** and **complete** set of inference rules
 - These are rules hold and all other rules that hold can be deduced from these

Inference Rules for FDs (2)

- Some additional inference rules that are useful:
 - **IR4 – Decomposition:**
If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$
 - **IR5 – Union:**
If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
 - **IR6 – Psuedotransitivity:**
If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$
- The last three inference rules, as well as any other inference rules, can be deduced from IR1, IR2, and IR3 (completeness property)

Inference Rules for FDs (3)

- **Closure** of a set F of FDs is the set F^+ of all FDs that can be inferred from F
- **Definition:**
 - For each such set of attributes X , we determine the set X^+ of attributes that are functionally determined by X based on F .
 - X^+ called the **closure of X under F** .
- X^+ can be calculated by repeatedly applying IR1, IR2, IR3 using the FDs in F

Algorithm 15.1 - Closure

- Determining X^+ , the closure of X under F
 $X^+ := X$;
 repeat
 $\text{old}X^+ := X^+$;
 for each functional dependency $Y \rightarrow Z$ in F do
 if $X^+ \supseteq Y$ then $X^+ := X^+ \cup Z$;
 until $(X^+ = \text{old}X^+)$,

Example: Determining X^+

$F = \{$ $f_1: \text{SSN} \rightarrow \text{EName},$
 $f_2: \text{PNumber} \rightarrow \{\text{PName}, \text{PLocation}\},$
 $f_3: \{\text{SSN}, \text{PNumber}\} \rightarrow \text{Hours}$
 $\}$
 $\{\text{SSN}, \text{PNumber}\}^+ = ?$

Example: Determining X^+ (2)

- $R(U, F)$

- $U = \{A, B, C, D, E, G, H\}$

- $F = \{A \rightarrow H, AB \rightarrow C, BC \rightarrow D, G \rightarrow B\}$

$X = \{A, B\}$. $X^+ = ?$

Equivalence of Sets of FDs

- Definition (**Covers**): F, G is set of functional dependencies
 - F **covers** G if every FD in G can be inferred from F
 - (i.e., if G^+ *subset-of* F^+)
- Two sets of FDs F and G are **equivalent** if:
 - Every FD in F can be inferred from G , and
 - Every FD in G can be inferred from F
 - Hence, F and G are equivalent if $F^+ = G^+$
- F and G are equivalent if F covers G and G covers F

Minimal Sets of FDs

- A set of FDs is **minimal** if it satisfies the following conditions:
 1. Every dependency in F has a single attribute for its right-hand-side.
 2. We cannot remove any dependency $X \rightarrow A$ in F with a dependency $Y \rightarrow A$, where Y is a proper subset of X , and still have a set of dependencies that is equivalent to F .
 3. We cannot remove any dependency from F and still have a set of dependencies that is equivalent to F .

Algorithm 15.2 - Minimal Cover

Finding a Minimal Cover F for a Set of Functional Dependencies E

Set $F := E$.

- Replace each functional dependency $X \rightarrow \{A_1, A_2, \dots, A_n\}$ in F by the n functional dependencies $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$.
- For each functional dependency $X \rightarrow A$ in F
for each attribute B that is an element of X
if $\{F - \{X \rightarrow A\}\} \cup \{(X - \{B\}) \rightarrow A\}$ is equivalent to F ,
then replace $X \rightarrow A$ with $(X - \{B\}) \rightarrow A$ in F .
- For each remaining functional dependency $X \rightarrow A$ in F
if $\{F - \{X \rightarrow A\}\}$ is equivalent to F ,
then remove $X \rightarrow A$ from F .

Example (1)

FDs of E: $\{B \rightarrow A, \quad (1)$

$D \rightarrow A, \quad (2)$

$AB \rightarrow D\} \quad (3)$

Finding the minimum cover of E?

Example (2)

FDs of F: { $A \rightarrow B$, (1)

$ABCD \rightarrow E$, (2)

$EF \rightarrow GH$, (3)

$ACDF \rightarrow EG$ (4)

Finding the minimum cover of F?

Minimal Sets of FDs (2)

- Every set of FDs has an equivalent minimal set
- There can be several equivalent minimal sets
 - If F has 2 minimum cover F' and F'' then $F' \equiv F''$
- There is no simple algorithm for computing a minimal set of FDs that is equivalent to a set F of FDs

Key of The Relational Schema

- Definition: A *superkey* of a relation schema R is set of attributes $K \subseteq R$ if given a relation $r(R)$ and any two tuples t_1 and t_2 in r then $t_1[K] \neq t_2[K]$.
- Definition: A *key* K of a relation schema R is a superkey and there is no proper subset of K that can be a key.

Key of The Relational Schema (2)

- Given a relation schema $R(U, F)$
list of attributes $U = \{ A_1, A_2, \dots, A_n \}$,
 F is a set of functional dependencies,
 K is a set of attributes and $K \subset R$.

We can say K is a key of E if the followings hold:

- $K \rightarrow U \in F^+$
- for all proper subset K' of $K (K' \subset K)$ then $K' \rightarrow U \notin F^+$

Key of The Relational Schema (3)

- The difference between key and superkey is that a key has to be minimal. That means if $K = \{ A_1, A_2, \dots, A_k \}$ then $K \setminus \{ A_i \}$ is not a key for R for any A_i
- If a relation schema has more than one key, each is called a ***candidate key***
- A candidate key will be chosen to be primary key.

Algorithm for finding a candidate key

- Input: Relation schema $R = \{ A_1, A_2, \dots, A_n \}$, set F of functional dependencies
- Output: A candidate key K for R .

$K = R$

for $i = 1$ to n do

{

$K_t = K - \{A_i\}$

if $K_t^+ = R$ then $K = K_t$

}

Example

- $R = ABCDEF$, $F = \{ BC \rightarrow AD, D \rightarrow E, CF \rightarrow B \}$.

Find a candidate key of this relation schema.

Q & A

